42 WAYS TO CONNECT POSTGRES — WHERE THE SERVER RUNS + HOW NESTJS TALKS TO IT

theory/41 is the one real build (Supabase + TypeORM). This file zooms out: every common way to get a Postgres server, and every common way for a NestJS app to talk to it. Two separate questions:

PART A — WHERE does the Postgres server live?	(changes only .env)
PART B — HOW does your code talk to it?	(changes the code)

Key idea: your NestJS code never cares WHERE Postgres runs. Swapping Supabase for local/Docker/Neon is just a different DATABASE_URL (plus SSL on/off). Swapping TypeORM for Prisma/raw pg is a code change.

THE CONNECTION STRING (same shape everywhere)

```
postgresql://<user>:<password>@<host>:<port>/<database>?<options>
```

user	**database user (postgres by default)**	
password	that user's password (URL-encode @ # / : ? %)	
host	localhost, a container name, or a cloud hostname	
port	5432 is Postgres' default	
database	which database inside the server	
options	e.g. ?sslmode=require	(force encrypted connection)

```
postgres://  and  postgresql://  mean the same thing.
```

Instead of one URL, most libraries also accept separate fields:

host, port, username, password, database Same information, just split up (useful when the password has many special characters).

PART A — WHERE THE SERVER RUNS

A1 — LOCAL INSTALL (Postgres directly on Windows)

HOW

```
postgresql.org -> Download -> Windows installer (EDB).
Installs: the Postgres server (runs as a Windows service),
```

```
          pgAdmin (GUI), psql (CLI). You set the postgres password
          during install.
```

Create a database for the app:

```
psql -U postgres
CREATE DATABASE nest_crud;
\q
```

.env:

```
DATABASE_URL="postgresql://postgres:mypass@localhost:5432/nest_crud"
```

SSL: off (ssl: false) — traffic never leaves your machine.

THEORY

- works offline, fastest (no network latency), free forever
- full control, see everything in pgAdmin
- it's installed on your PC permanently, uses RAM as a service
- only your machine can reach it

A2 — DOCKER (Postgres inside a container)

HOW

```
docker run --name pg-dev -e POSTGRES_PASSWORD=devpass -e POSTGRES_DB=nest_crud -p 5432:5432 -d postgres

docker ps                          // is it running?
docker stop pg-dev / docker start pg-dev
docker exec -it pg-dev psql -U postgres -d nest_crud    // SQL shell
```

Keep data even if the container is deleted (a VOLUME): docker run ... -v pgdata:/var/lib/postgresql/data ... postgres

docker-compose.yml version (start with `docker compose up -d`): services: db: image: postgres:17 environment: POSTGRES_PASSWORD: devpass POSTGRES_DB: nest_crud ports: ["5432:5432"] volumes: [pgdata:/var/lib/postgresql/data] volumes: pgdata:

.env:

```
DATABASE_URL="postgresql://postgres:devpass@localhost:5432/nest_crud"
```

SSL: off.

THEORY

```
-p 5432:5432   maps your PC's port 5432 to the container's port 5432,
               so the app connects to "localhost" as if it were local
-e ...         env vars the official postgres image reads on first start
+ nothing installed on Windows, delete it in one command
+ exact same version for everyone on the team
```

```
- needs Docker Desktop running
- without a volume, data is lost when the container is removed
```

A3 — HOSTED / CLOUD (someone else runs the server)

```
Supabase       free tier, dashboard + Table Editor, pooler URI
               (theory/41)
Neon           free tier, "serverless" Postgres, branches of a DB
               like git branches
Railway        click "New -> PostgreSQL", copy the URI
Render         free Postgres (expires after a period on free plan)
Aiven          managed Postgres, free small plan
AWS RDS / Aurora, Google Cloud SQL, Azure Database for PostgreSQL
               production-grade, paid, backups, scaling, private networks
```

.env examples:

Supabase	postgresql://postgres.<ref>:<pass>@aws-0-<region>.pooler.supabase.com:5432/postgres
Neon	postgresql://<user>:<pass>@ep-xxx.<region>.aws.neon.tech/neondb?sslmode=require
RDS	postgresql://<user>:<pass>@mydb.xxxx.<region>.rds.amazonaws.com:5432/nest_crud

SSL: ON — data travels over the internet, providers require it:

```
ssl: { rejectUnauthorized: false }      // encrypted, cert not verified (learning)
ssl: { ca: fs.readFileSync('ca.pem') }   // encrypted + verified (production)
```

THEORY

- no install, reachable from anywhere (deployed apps, teammates)
- backups, dashboards, monitoring included
- needs internet, slower than local (every query is a network trip)
- free tiers pause/limit; credentials must be kept secret

A4 — CONNECTION POOLERS (PgBouncer, Supavisor)

A pooler sits BETWEEN the app and Postgres and shares a few real connections among many clients. Postgres can only handle a limited number of connections (each one is a process on the server).

```
  App --> pooler (many cheap connections) --> Postgres (few real ones)

Session mode     one real connection for your whole session —
                 right choice for a long-running NestJS server
Transaction mode connection returned after each transaction — for
                 serverless functions; breaks prepared statements
```

Supabase's pooler.supabase.com:5432 (session) / :6543 (transaction) is exactly this.

A5 — SWITCHING BETWEEN THEM WITHOUT CODE CHANGES

```
.env (local dev):
  DATABASE_URL="postgresql://postgres:devpass@localhost:5432/nest_crud"
  DB_SSL=false
.env (Supabase):
  DATABASE_URL="postgresql://postgres.<ref>:<pass>@...pooler.supabase.com:5432/postgres"
  DB_SSL=true
```

app.module.ts:

```
url: configService.getOrThrow<string>('DATABASE_URL'),
ssl: configService.get('DB_SSL') === 'true' ? { rejectUnauthorized: false } : false,
```

Same code, different .env — that is the whole point of env vars (theory/28, theory/29).

PART B — HOW NESTJS TALKS TO POSTGRES

All of these sit on top of the `pg` driver (except where noted): your code → library → pg driver → TCP → Postgres

B1 — TYPEORM (what this project uses)

```
npm install @nestjs/typeorm typeorm pg

// app.module.ts
TypeOrmModule.forRootAsync({
  inject: [ConfigService],
  useFactory: (c: ConfigService) => ({
    type: 'postgres',
    url: c.getOrThrow<string>('DATABASE_URL'),
    autoLoadEntities: true,
    synchronize: true, // dev only
  }),
})

// entity = table
@Entity('students') export class Student { @PrimaryGeneratedColumn() id!: number; @Column() name!:
string; }

// service
constructor(@InjectRepository(Student) private repo: Repository<Student>) {}
this.repo.find();  this.repo.findOneBy({ id });  this.repo.save(x);  this.repo.delete(id);
```

THEORY

- official Nest package, decorators feel like Mongoose @Schema
- synchronize for quick dev, migrations for prod
- complex queries need QueryBuilder; some typing gaps

B2 — PRISMA

```
npm install prisma --save-dev
npm install @prisma/client
npx prisma init          // creates prisma/schema.prisma + DATABASE_URL in .env

// prisma/schema.prisma
datasource db { provider = "postgresql"  url = env("DATABASE_URL") }
generator client { provider = "prisma-client-js" }
model Student {
  id     Int     @id @default(autoincrement())
  name   String
  age    Int
  email  String  @unique
  course String?
}

npx prisma migrate dev --name init   // creates the table + generates the client

// prisma.service.ts
@Injectable()
export class PrismaService extends PrismaClient implements OnModuleInit {
  async onModuleInit() { await this.$connect(); }
}

// student service
constructor(private prisma: PrismaService) {}
this.prisma.student.findMany();
this.prisma.student.findUnique({ where: { id } });
this.prisma.student.create({ data: dto });
this.prisma.student.update({ where: { id }, data: dto });
this.prisma.student.delete({ where: { id } });
```

THEORY

- schema file is the single source of truth, fully typed client (autocomplete on every field), excellent migrations
- Prisma Studio (npx prisma studio) = GUI for your data
- extra generate step; uses its own query engine, not plain classes

B3 — DRIZZLE ORM

```
npm install drizzle-orm pg
npm install -D drizzle-kit

// schema.ts
export const students = pgTable('students', {
  id: serial('id').primaryKey(),
  name: text('name').notNull(),
  email: text('email').notNull().unique(),
});

const db = drizzle(new Pool({ connectionString: process.env.DATABASE_URL }));
await db.select().from(students).where(eq(students.id, 1));
await db.insert(students).values(dto).returning();
```

THEORY

- looks like SQL, very light, great TypeScript types
- no official @nestjs package — you wrap it in your own provider

B4 — MIKRO-ORM

```
npm install @mikro-orm/core @mikro-orm/postgresql @mikro-orm/nestjs

MikroOrmModule.forRoot({ driver: PostgreSqlDriver, clientUrl: process.env.DATABASE_URL,
autoLoadEntities: true })
@Entity() export class Student { @PrimaryKey() id!: number; @Property() name!: string; }
em.find(Student, {});  em.persistAndFlush(student);
```

THEORY

- "unit of work": collects changes and writes them in one go
- official Nest package
- smaller community than TypeORM/Prisma

B5 — SEQUELIZE

```
npm install @nestjs/sequelize sequelize sequelize-typescript pg pg-hstore

SequelizeModule.forRoot({ dialect: 'postgres', uri: process.env.DATABASE_URL, autoLoadModels: true,
synchronize: true })
@Table export class Student extends Model { @Column name!: string; }
@InjectModel(Student) private studentModel: typeof Student
this.studentModel.findAll();  this.studentModel.create(dto);
```

THEORY

- oldest Node ORM, lots of tutorials, official Nest package
- weaker TypeScript support than the newer options

B6 — KNEX (query builder, not an ORM)

```
npm install knex pg

const db = knex({ client: 'pg', connection: process.env.DATABASE_URL });
await db('students').select('*');
await db('students').where({ id: 1 }).first();
await db('students').insert(dto).returning('*');
```

THEORY

- you build queries with JS, no entity classes, good migrations
- results are plain objects, little type safety

B7 — RAW `pg` DRIVER (write the SQL yourself)

```
npm install pg
npm install -D @types/pg

// pg.provider.ts — one shared Pool for the whole app
export const PG_POOL = 'PG_POOL';
export const pgPoolProvider = {
  provide: PG_POOL,
  inject: [ConfigService],
  useFactory: (c: ConfigService) =>
    new Pool({ connectionString: c.getOrThrow('DATABASE_URL'), ssl: { rejectUnauthorized: false } }),
};

// service
constructor(@Inject(PG_POOL) private pool: Pool) {}

async findAll() {
  const { rows } = await this.pool.query('SELECT * FROM students');
  return rows;
}
async findOne(id: number) {
  const { rows } = await this.pool.query('SELECT * FROM students WHERE id = $1', [id]);
  if (!rows[0]) throw new NotFoundException();
  return rows[0];
}
async create(dto: CreateStudentDto) {
  const { rows } = await this.pool.query(
    'INSERT INTO students (name, age, email, course) VALUES ($1, $2, $3, $4) RETURNING *',
    [dto.name, dto.age, dto.email, dto.course],
  );
  return rows[0];
}
```

Pool vs Client:

Client	one single connection — connect(), query, end()
Pool	keeps several connections open and reuses them — use this in a server (many requests at once)

```
$1, $2 placeholders: ALWAYS pass values as the second argument,
NEVER build SQL with string concatenation:
  BAD   'SELECT * FROM students WHERE id = ' + id     // SQL injection
  GOOD  'SELECT * FROM students WHERE id = $1', [id]
```

THEORY

- full control, fastest, nothing hidden, best way to learn SQL
- you write CREATE TABLE yourself, no auto types, more code

B8 — GUI / CLI TOOLS (connect as a human, not from code)

psql	official CLI:	psql "postgresql://user:pass@host:5432/db"
pgAdmin	official GUI, comes with the Windows installer	
DBeaver	free GUI for many databases	
TablePlus	clean GUI (paid, free trial)	
VS Code	"PostgreSQL" / "SQLTools" extensions	
Supabase / Neon dashboards		built-in table editor + SQL editor

All of them take the SAME connection details as your app.

COMPARISON

WHERE	Setup	Cost	Internet	SSL	Best for
Local install	installer	free	no	off	offline dev
Docker	1 command	free	no	off	dev, teams
Supabase/Neon	sign up	free tier	yes	on	learning, small apps
RDS/Cloud SQL	console	paid	yes	on	production
HOW	Style	Nest package		Typing	Learning curve
TypeORM	entities/repo	@nestjs/typeorm		good	medium
Prisma	schema + client	(custom service)		best	easy
Drizzle	SQL-like TS	(custom provider)		great	medium
MikroORM	unit of work	@mikro-orm/nestjs		good	medium-high
Sequelize	models	@nestjs/sequelize		ok	easy
Knex	query builder	(custom provider)		weak	easy
raw pg	SQL strings	(custom provider)		manual	needs SQL

★ RULE OF THUMB

WHERE the server runs only changes DATABASE_URL (+ SSL on for cloud, off for local). HOW you talk to it is the library choice: TypeORM if you like decorators (same feel as Mongoose), Prisma for the best types and migrations, raw pg to really learn SQL. Whatever you pick: keep the URL in .env, use one shared pool, always use $1 parameters, and never use synchronize in production.

See also: **theory/40-postgres-end-to-end.txt** (generic setup), **theory/41-supabase-postgres-crud-full-steps.txt** (the real build), **theory/36-postgres-setup-and-crud.txt**, **theory/37-sql-cheatsheet.txt**, **theory/28-environment-variables.txt**, **theory/29-nestjs-config-module.txt**